```
expected ';' before…

expected ')' before…

expected '}' before…
```

The Arduino development environment will generally give you a pretty good idea where to look for this problem by moving the cursor near the problematic code and highlighting the code near where the error was found, so you can fix it by replacing the missing character before trying to verify the code again.

■ **Note** Remember that at the end of every simple statement a semicolon is needed to separate one statement from the next. Also, all parenthesis, curly braces, and block comments need to be matched in pairs.

Many syntax errors stem from Arduino C being case sensitive. In a **case sensitive** language, whether a letter is uppercase or lowercase must match exactly between what is entered and the syntax of the function, variable, or other statement being used. When writing Arduino sketches, there is a big difference between `pinmode()`, `pinMode()`, and `PINMODE()`—only one of these will actually work. The following are some examples of errors relating to case:

```
'pinmode' was not declared in this scope

'high' was not declared in this scope
```

These syntactical mistakes are examples of using the wrong case commonly when calling functions or using variables or constants. Look in the highlighted line of code for where you might have mistakenly typed `high` instead of `HIGH`, `output` instead of `OUTPUT`, `pinmode` instead of `pinMode`, or any other instances that are case sensitive.

No matter the error, just take a breath and think it through. I'm sure you will find it in no time. Writing code has a funny way of needing to be extremely logical and literal. If you are still having problems getting it going, you might turn to the generally helpful Arduino Forums at `http://arduino.cc/forum`.

Summary

Whew! That was a lot to get through for our first project, but we made it! We started with the philosophy of sketching in code; introduced our first project, including some diagrams and source code; discussed the structure of Arduino C, including using comments, the basics of Arduino functions, and working with statements and the elements of syntax; and then we wrapped things up by looking at compiling, saving, and uploading our sketches, as well as some common errors that might crop up once in a while.